

# Documentação de Arquitetura

diagnostico-medico-api — Projeto 1: Otimização de Modelos de Diagnóstico

---

## 1. Contexto e Escopo

Este documento descreve a arquitetura de infraestrutura, deployment e observabilidade implementada para a API diagnostico-medico-api, desenvolvida no âmbito do Tech Challenge Fase 2, Projeto 1 (Otimização de Modelos de Diagnóstico), conforme especificado no documento do desafio da IADT.

O foco deste documento é a arquitetura de nuvem, o pipeline de CI/CD e os mecanismos de escalabilidade, resiliência e monitoramento da solução — respondendo diretamente ao requisito obrigatório "Configurar recursos de escalabilidade automática para lidar com variações de demanda", incluindo monitoramento, logging e documentação de arquitetura.

*Nota de escopo: a otimização de hiperparâmetros via Algoritmos Genéticos e a integração com LLMs para interpretação clínica dos diagnósticos (requisitos 1 e 3 do desafio) são tratadas em documento técnico específico do modelo de ML, não fazendo parte deste documento de arquitetura de infraestrutura.*

## 2. Visão Geral da Arquitetura

A solução foi implementada como uma API FastAPI containerizada, publicada em AWS ECS Fargate por trás de um Application Load Balancer, com provisionamento totalmente via Terraform (estado remoto em Terraform Cloud) e deploy automatizado via GitHub Actions com autenticação OIDC (sem chaves de longa duração). O diagrama abaixo resume o fluxo entre os componentes.

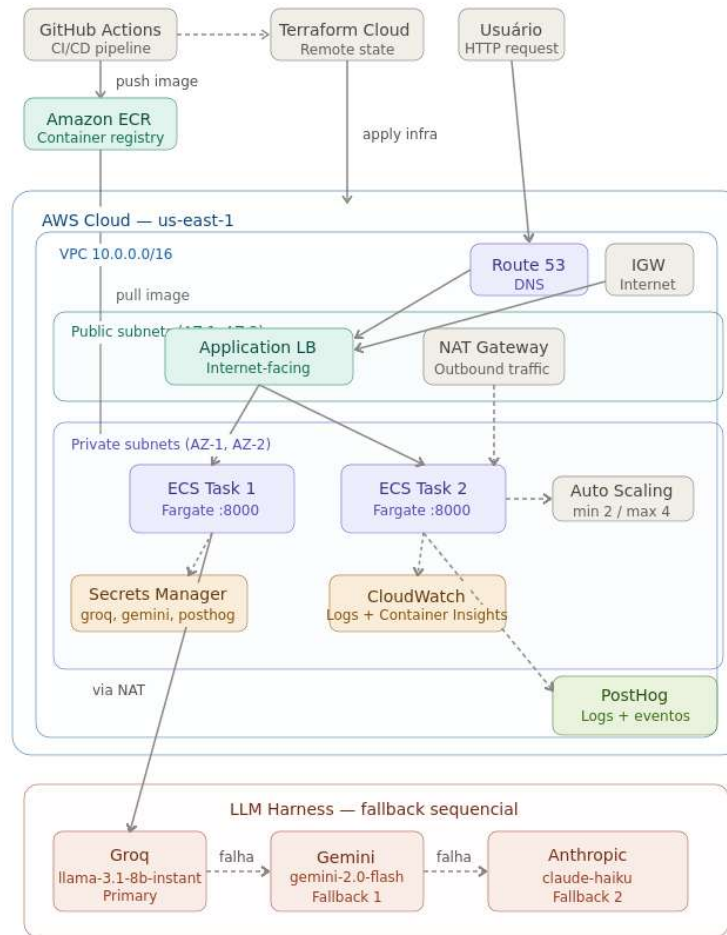


Figura 1 — Arquitetura AWS do diagnostico-medico-api

## 3. Componentes da Arquitetura

### 3.1 Pipeline de CI/CD (GitHub Actions)

O deploy é orquestrado por um workflow único (`.github/workflows/pipeline.yml`) disparado por push, pull request e, para as etapas de publicação e deploy, por `workflow_dispatch` com um parâmetro de ação (`push`, `deploy` ou `full`). O pipeline segue quatro estágios sequenciais:

- **Lint** — `ruff check` e `ruff format --check` sobre todo o código-fonte;
- **Test** — execução da suíte `pytest` com relatório de cobertura (`--cov=app`), publicando o artefato de cobertura;
- **Build** — build da imagem Docker (multi-stage), tagueada com o SHA do commit e latest;
- **Push/Deploy** — autenticação na AWS via OIDC (`aws-actions/configure-aws-credentials`, sem secrets estáticos), push da imagem para o ECR e, na etapa de deploy, `terraform init/plan/apply` seguido de `aws ecs wait services-stable` para confirmar a estabilização do serviço.

A autenticação entre GitHub Actions e AWS usa um provider OIDC (`token.actions.githubusercontent.com`) associado a uma IAM Role (`GitHub-Actions-Deploy`) restrita ao repositório `MacielMjose/diagnostico-medico-api` via condição `StringLike` no `claim sub`, eliminando a necessidade de credenciais de longa duração armazenadas como secrets.

### 3.2 Infraestrutura como Código (Terraform)

Todo o provisionamento é declarado em Terraform, com estado remoto gerenciado no Terraform Cloud (workspace `diagnostico-medico-api-dev`), permitindo colaboração e histórico de state sem depender de backend local ou S3/DynamoDB self-managed.

**Rede:** VPC dedicada com subnets públicas e privadas distribuídas em múltiplas zonas de disponibilidade, Internet Gateway para as subnets públicas e NAT Gateway (um por subnet pública, com Elastic IP dedicado) para permitir que as tarefas Fargate nas subnets privadas acessem a internet (ex.: chamadas às APIs de LLM) sem exposição direta de IP público.

**Roteamento:** route tables públicas apontando para o Internet Gateway e route tables privadas apontando para o NAT Gateway correspondente, associadas por zona de disponibilidade.

**Segurança:** dois security groups — um para o ALB (ingress 80/443 aberto ao público) e um para as tarefas ECS (ingress restrito à porta do container e somente a partir do security group do ALB), com egress irrestrito para permitir chamadas de saída às APIs de LLM.

### 3.3 Camada de Computação (ECS Fargate)

O cluster ECS roda em modo serverless (Fargate), com Container Insights habilitado para métricas nativas de CPU/memória por tarefa. A estratégia de capacity provider prioriza Fargate on-demand (peso 100) com Fargate Spot disponível como opção de custo (peso 0 por padrão).

- Task Definition: define CPU/memória do container, variáveis de ambiente (incluindo o provider de LLM ativo e a cadeia de fallback), segredos injetados via Secrets Manager, e um health check em nível de container (curl em `/health`) com intervalo, timeout e retries configuráveis;
- IAM: duas roles distintas — execution role (permissões de pull no ECR e escrita no CloudWatch Logs, além de leitura dos secrets) e task role (permissões em runtime da aplicação), seguindo o princípio de menor privilégio;
- Networking: as tarefas rodam nas subnets privadas, sem IP público, e recebem tráfego apenas através do Application Load Balancer;
- Service: mantém o desired count de tarefas, associado ao target group do ALB, com dependência explícita do listener e das policies IAM antes do provisionamento.

### 3.4 Balanceamento de Carga (ALB)

Um Application Load Balancer público, posicionado nas subnets públicas, distribui o tráfego HTTP (porta 80) para o target group das tarefas ECS via IP. O target group realiza health checks periódicos no endpoint `/health` (2 checagens saudáveis/não saudáveis para troca de estado, intervalo de 30s), garantindo que apenas tarefas saudáveis recebam requisições. Um listener HTTPS com certificado ACM está preparado no código (comentado), pronto para ser habilitado quando um domínio e certificado estiverem configurados.

### 3.5 Escalabilidade Automática (Auto Scaling)

Para atender ao requisito de "lidar com variações de demanda", o serviço ECS possui um Application Auto Scaling Target configurado com três políticas de target tracking independentes, permitindo que o número de tarefas suba ou desça automaticamente conforme a carga real:

| Política             | Métrica-alvo                       | Objetivo                           |
|----------------------|------------------------------------|------------------------------------|
| CPU                  | ECSServiceAverageCPUUtilization    | 70% de utilização média de CPU     |
| Memória              | ECSServiceAverageMemoryUtilization | 80% de utilização média de memória |
| Requisições por alvo | ALBRequestCountPerTarget           | 1000 requisições por tarefa        |

As três políticas atuam em conjunto sobre o mesmo `aws_appautoscaling_target`, com `min_capacity` e `max_capacity` parametrizados por variável — permitindo, por exemplo, reduzir a capacidade mínima a zero/um durante períodos ociosos, o que é coerente com a prática adotada no projeto de derrubar a infraestrutura quando não está em uso e reprovisioná-la antes da avaliação, para controle de custos de crédito AWS.

### 3.6 Gestão de Segredos (AWS Secrets Manager)

As chaves de API dos provedores de LLM e demais credenciais sensíveis são armazenadas no AWS Secrets Manager, com nomenclatura padronizada `app-name/environment/secret-name`, e injetadas diretamente nas variáveis de ambiente do container em runtime (bloco `secrets` da task definition) — nunca hardcoded na imagem ou no código-fonte. A `recovery_window_in_days` de 7 dias permite restaurar um segredo apagado por engano antes da exclusão definitiva.

### 3.7 Observabilidade (CloudWatch + PostHog/OpenTelemetry)

A observabilidade combina dois canais complementares:

- CloudWatch Logs — log group dedicado (`/ecs/<app_name>`) com retenção configurável, recebendo a saída padrão do container via driver `awslogs`; cobre logs de infraestrutura básicos e é a fonte usada pelo próprio ECS/Fargate;
- PostHog via OpenTelemetry — dois fluxos distintos e documentados no próprio código: (1) `capture_*` envia eventos de produto (ex.: `prediction_success`, `llm_success`) para a aba Activity do PostHog; (2) o logger estruturado (`structlog`) é conectado a um `LoggingHandler` do OpenTelemetry, que exporta via OTLP HTTP para a aba Logs do PostHog — permitindo correlacionar comportamento de produto e logs técnicos na mesma ferramenta.

Essa separação deliberada entre eventos de produto e logs técnicos foi uma decisão de design registrada nos comentários do próprio módulo de monitoramento, evitando que as duas naturezas de dado se misturem em uma única visualização.

### 3.8 Resiliência de LLM — Harness com Fallback Sequencial

Para resolver os problemas de exaustão de memória e timeout observados ao rodar um LLM local (Ollama/llama3.2) dentro do container Fargate, a solução adotada foi externalizar a chamada ao LLM para provedores gerenciados, com um harness de fallback sequencial: Groq (primário) → Gemini → Anthropic Claude Haiku. O `LLMHarnessProvider` tenta cada provedor da lista em ordem; se um provedor falhar (exceção de qualquer tipo), o harness registra a falha via log estruturado e tenta o próximo, só propagando erro (`LLMFallbackError`) se todos os provedores da cadeia falharem. Cada tentativa e seu resultado (sucesso, uso de fallback, tipo de erro) são logados de forma estruturada, alimentando diretamente o canal de observabilidade descrito no item 3.7.

## 4. Fluxo de Deploy Ponta a Ponta

1. Push de código dispara lint e testes automaticamente em qualquer branch;
2. Um `workflow_dispatch` manual com ação push constrói a imagem Docker e a publica no ECR (autenticado via OIDC), tageada com `latest` e com o SHA do commit;
3. Um `workflow_dispatch` com ação `deploy` (ou `full`) executa `terraform init/plan/apply` contra o workspace no Terraform Cloud, criando ou atualizando a infraestrutura declarada;
4. Ao final do `apply`, o pipeline aguarda ativamente (`aws ecs wait services-stable`) até que o novo serviço ECS esteja estável antes de considerar o deploy concluído;
5. Como a infraestrutura é derrubada quando ociosa (`terraform destroy`) e reprovisionada sob demanda antes da avaliação, esse fluxo de `workflow_dispatch` parametrizado é o que permite reconstruir o ambiente de forma repetível e controlada, sem custo contínuo de créditos AWS.

## 5. Decisões de Arquitetura e Trade-offs

| Decisão                                                        | Motivação                                                                          | Trade-off aceito                                                                          |
|----------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| LLM externo (Groq/Gemini/Claude) em vez de Ollama in-container | Ollama causava exaustão de memória e timeout mesmo escalando recursos do container | Dependência de APIs externas e de rede via NAT Gateway; mitigado pelo harness de fallback |
| Terraform Cloud como backend de state                          | Colaboração e histórico de state sem gerenciar S3/DynamoDB manualmente             | Dependência de um serviço externo (HashiCorp) para aplicar mudanças de infra              |
| OIDC em vez de chaves de acesso estáticas no GitHub            | Elimina secrets de longa duração e reduz superfície de ataque                      | Configuração inicial mais complexa (provider OIDC + trust)                                |

|                                                                      |                                                                       |                                                                                               |
|----------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Actions                                                              |                                                                       | policy)                                                                                       |
| Infra provisionada sob demanda (destroy/apply via workflow_dispatch) | Controle de custo de créditos AWS durante o desenvolvimento acadêmico | Cold start antes de cada avaliação; exige que o pipeline de deploy seja confiável e repetível |

## 6. Referência Rápida de Recursos AWS

| Camada            | Recursos Terraform                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| Rede              | aws_vpc, aws_subnet (público/privado), aws_internet_gateway, aws_nat_gateway, aws_eip, aws_route_table(s) |
| Segurança         | aws_security_group (ALB e ECS tasks), IAM roles (execution, task, GitHub Actions)                         |
| Computação        | aws_ecs_cluster, aws_ecs_cluster_capacity_providers, aws_ecs_task_definition, aws_ecs_service             |
| Rede de aplicação | aws_lb, aws_lb_target_group, aws_lb_listener                                                              |
| Escalabilidade    | aws_appautoscaling_target, aws_appautoscaling_policy (CPU, memória, requisições/ALB)                      |
| Segredos e logs   | aws_secretsmanager_secret, aws_cloudwatch_log_group                                                       |
| Imagens           | aws_ecr_repository                                                                                        |
| CI/CD             | aws_iam_openid_connect_provider (data source), aws_iam_role.github_actions                                |

## 7. Panorama Geral da Solução

A arquitetura implementada resolve o requisito de escalabilidade automática do desafio combinando três frentes que se reforçam mutuamente: infraestrutura elástica, resiliência de dependências externas e observabilidade de dois níveis.

**Elasticidade real, não apenas configurada:** as três políticas de Auto Scaling (CPU, memória e requisições por alvo) atuam em conjunto sobre o mesmo serviço ECS, permitindo que a capacidade acompanhe tanto picos de carga computacional quanto picos de tráfego HTTP, sem intervenção manual.

**Resiliência onde ela mais importa:** o LLM Harness com fallback sequencial (Groq → Gemini → Claude) evita que uma falha pontual em um provedor externo derrube a funcionalidade de interpretação clínica, e a arquitetura de rede (subnets privadas + NAT Gateway) garante que essas chamadas saiam de forma controlada, sem expor as tarefas Fargate diretamente à internet.

**Segurança por padrão:** autenticação OIDC no pipeline (sem credenciais estáticas), segredos centralizados no Secrets Manager e segmentação de rede via security groups reduzem a superfície de ataque em cada camada da solução.

**Observabilidade acionável:** a separação entre eventos de produto (PostHog Activity) e logs técnicos (CloudWatch e PostHog Logs via OpenTelemetry) permite diagnosticar tanto problemas de infraestrutura quanto o comportamento real da API em produção, sem misturar as duas naturezas de dado.

**Custo sob controle:** o modelo de infraestrutura provisionada sob demanda — apply antes da avaliação, destroy quando ociosa — mostra que decisões de arquitetura também foram tomadas considerando o contexto de um projeto acadêmico com créditos de nuvem limitados, sem abrir mão de uma infraestrutura fiel a um ambiente produtivo real.

Em conjunto, essas escolhas entregam uma base de infraestrutura que atende ao requisito obrigatório de escalabilidade automática e monitoramento do desafio, ao mesmo tempo em que permanece deliberadamente simples de reconstruir, auditar e operar.

---